

在文件中名字的出现处对其进行解析

287

当成员定义在类的外部时，名字查找的第三步不仅要考虑类定义之前的全局作用域中的声明，还需要考虑在成员函数定义之前的全局作用域中的声明。例如：

```
int height; // 定义了一个名字，稍后将在 Screen 中使用
class Screen {
public:
    typedef std::string::size_type pos;
    void setHeight(pos);
    pos height = 0; // 隐藏了外层作用域中的 height
};
Screen::pos verify(Screen::pos);
void Screen::setHeight(pos var) {
    // var: 参数
    // height: 类的成员
    // verify: 全局函数
    height = verify(var);
}
```

请注意，全局函数 `verify` 的声明在 `Screen` 类的定义之前是不可见的。然而，名字查找的第三步包括了成员函数出现之前的全局作用域。在此例中，`verify` 的声明位于 `setHeight` 的定义之前，因此可以被正常使用。

7.4.1 节练习

练习 7.34: 如果我们把第 256 页 `Screen` 类的 `pos` 的 `typedef` 放在类的最后一行会发生什么情况？

练习 7.35: 解释下面代码的含义，说明其中的 `Type` 和 `initVal` 分别使用了哪个定义。如果代码存在错误，尝试修改它。

```
typedef string Type;
Type initVal();
class Exercise {
public:
    typedef double Type;
    Type setVal(Type);
    Type initVal();
private:
    int val;
};
Type Exercise::setVal(Type parm) {
    val = parm + initVal();
    return val;
}
```

7.5 构造函数再探

288

对于任何 C++ 的类来说，构造函数都是其中重要的组成部分。我们已经在 7.1.4 节（第 235 页）中介绍了构造函数的基础知识，本节将继续介绍构造函数的一些其他功能，并对

之前已经介绍的内容进行一些更深入的讨论。



7.5.1 构造函数初始值列表

当我们定义变量时习惯于立即对其进行初始化，而非先定义、再赋值：

```
string foo = "Hello World!";           // 定义并初始化
string bar;                             // 默认初始化成空 string 对象
bar = "Hello World!";                   // 为 bar 赋一个新值
```

就对象的数据成员而言，初始化和赋值也有类似的区别。如果没有在构造函数的初始值列表中显式地初始化成员，则该成员将在构造函数体之前执行默认初始化。例如：

```
// Sales_data 构造函数的一种写法，虽然合法但比较草率：没有使用构造函数初始值
Sales_data::Sales_data(const string &s,
                        unsigned cnt, double price)
{
    bookNo = s;
    units_sold = cnt;
    revenue = cnt * price;
}
```

这段代码和我们在 237 页的原始定义效果是相同的：当构造函数完成后，数据成员的值相同。区别是原来的版本初始化了它的数据成员，而这个版本是对数据成员执行了赋值操作。这一区别到底会有什么深层次的影响完全依赖于数据成员的类型。

构造函数的初始值有时必不可少

有时我们可以忽略数据成员初始化和赋值之间的差异，但并非总能这样。如果成员是 `const` 或者是引用的话，必须将其初始化。类似的，当成员属于某种类型且该类没有定义默认构造函数时，也必须将这个成员初始化。例如：

```
class ConstRef {
public:
    ConstRef(int ii);
private:
    int i;
    const int ci;
    int &ri;
};
```

289 和其他常量对象或者引用一样，成员 `ci` 和 `ri` 都必须被初始化。因此，如果我们没有为它们提供构造函数初始值的话将引发错误：

```
// 错误：ci 和 ri 必须被初始化
ConstRef::ConstRef(int ii)
{ // 赋值：
    i = ii;                       // 正确
    ci = ii;                       // 错误：不能给 const 赋值
    ri = i;                       // 错误：ri 没被初始化
}
```

随着构造函数体一开始执行，初始化就完成了。我们初始化 `const` 或者引用类型的数据成员的唯一机会就是通过构造函数初始值，因此该构造函数的正确形式应该是：

```
// 正确：显式地初始化引用和 const 成员
ConstRef::ConstRef(int ii): i(ii), ci(ii), ri(i) { }
```



如果成员是 const、引用，或者属于某种未提供默认构造函数的类类型，我们必须通过构造函数初始值列表为这些成员提供初值。

建议：使用构造函数初始值

在很多类中，初始化和赋值的区别事关底层效率问题：前者直接初始化数据成员，后者则先初始化再赋值。

除了效率问题外更重要的是，一些数据成员必须被初始化。建议读者养成使用构造函数初始值的习惯，这样能避免某些意想不到的编译错误，特别是遇到有的类含有需要构造函数初始值的成员时。

成员初始化的顺序

显然，在构造函数初始值中每个成员只能出现一次。否则，给同一个成员赋两个不同的初始值有什么意义呢？

不过让人稍感意外的是，构造函数初始值列表只说明用于初始化成员的值，而不限定初始化的具体执行顺序。

成员的初始化顺序与它们在类定义中的出现顺序一致：第一个成员先被初始化，然后第二个，以此类推。构造函数初始值列表中初始值的前后位置关系不会影响实际的初始化顺序。

一般来说，初始化的顺序没什么特别要求。不过如果一个成员是用另一个成员来初始化的，那么这两个成员的初始化顺序就很关键了。

举个例子，考虑下面这个类：

```
class X {
    int i;
    int j;
public:
    // 未定义的：i 在 j 之前被初始化
    X(int val): j(val), i(j) { }
};
```

290

在此例中，从构造函数初始值的形式上来看仿佛是用 val 初始化了 j，然后再用 j 初始化 i。实际上，i 先被初始化，因此这个初始值的效果是试图使用未定义的值 j 初始化 i！

有的编译器具备一项比较友好的功能，即当构造函数初始值列表中的数据成员顺序与这些成员声明的顺序不符时会生成一条警告信息。



最好令构造函数初始值的顺序与成员声明的顺序保持一致。而且如果可能的话，尽量避免使用某些成员初始化其他成员。

如果可能的话，最好用构造函数的参数作为成员的初始值，而尽量避免使用同一个对

象的其他成员。这样的好处是我们可以不必考虑成员的初始化顺序。例如，x 的构造函数如果写成如下的形式效果会更好：

```
X(int val): i(val), j(val) { }
```

在这个版本中，i 和 j 初始化的顺序就没什么影响了。

默认实参和构造函数

Sales_data 默认构造函数的行为与只接受一个 string 实参的构造函数差不多。唯一的区别是接受 string 实参的构造函数使用这个实参初始化 bookNo，而默认构造函数（隐式地）使用 string 的默认构造函数初始化 bookNo。我们可以把它们重写成一个使用默认实参（参见 6.5.1 节，第 211 页）的构造函数：

```
class Sales_data {
public:
    // 定义默认构造函数，令其与只接受一个 string 实参的构造函数功能相同
    Sales_data(std::string s = ""): bookNo(s) { }
    // 其他构造函数与之前一致
    Sales_data(std::string s, unsigned cnt, double rev):
        bookNo(s), units_sold(cnt), revenue(rev*cnt) { }
    Sales_data(std::istream &is) { read(is, *this); }
    // 其他成员与之前的版本一致
};
```

在上面这段程序中，类的接口与第 237 页的代码是一样的。当没有给定实参，或者给定了一个 string 实参时，两个版本的类创建了相同的对象。因为我们不提供实参也能调用上述的构造函数，所以该构造函数实际上为我们的类提供了默认构造函数。

291



如果一个构造函数为所有参数都提供了默认实参，则它实际上也定义了默认构造函数。

值得注意的是，我们不应该为 Sales_data 接受三个实参的构造函数提供默认值。因为如果用户为售出书籍的数量提供了一个非零的值，则我们会期望用户同时提供这些书籍的售出价格。

7.5.1 节练习

练习 7.36: 下面的初始值是错误的，请找出问题所在并尝试修改它。

```
struct X {
    X (int i, int j): base(i), rem(base % j) { }
    int rem, base;
};
```

练习 7.37: 使用本节提供的 Sales_data 类，确定初始化下面的变量时分别使用了哪个构造函数，然后罗列出每个对象所有数据成员的值。

```
Sales_data first_item(cin);

int main() {
    Sales_data next;
    Sales_data last("9-999-99999-9");
}
```

练习 7.38: 有些情况下我们希望提供 `cin` 作为接受 `istream&` 参数的构造函数的默认实参, 请声明这样的构造函数。

练习 7.39: 如果接受 `string` 的构造函数和接受 `istream&` 的构造函数都使用默认实参, 这种行为合法吗? 如果不, 为什么?

练习 7.40: 从下面的抽象概念中选择一个 (或者你自己指定一个), 思考这样的类需要哪些数据成员, 提供一组合理的构造函数并阐明这样做的原因。

- | | | |
|-------------|------------|--------------|
| (a) Book | (b) Date | (c) Employee |
| (d) Vehicle | (e) Object | (f) Tree |

7.5.2 委托构造函数

C++11 新标准扩展了构造函数初始值的功能, 使得我们可以定义所谓的**委托构造函数** (delegating constructor)。一个委托构造函数使用它所属类的其他构造函数执行它自己的初始化过程, 或者说它把它自己的一些 (或者全部) 职责委托给了其他构造函数。

C++
11

和其他构造函数一样, 一个委托构造函数也有一个成员初始值的列表和一个函数体。在委托构造函数内, 成员初始值列表只有一个唯一的入口, 就是类名本身。和其他成员初始值一样, 类名后面紧跟圆括号括起来的参数列表, 参数列表必须与类中另外一个构造函数匹配。

292

举个例子, 我们使用委托构造函数重写 `Sales_data` 类, 重写后的形式如下所示:

```
class Sales_data {
public:
    // 非委托构造函数使用对应的实参初始化成员
    Sales_data(std::string s, unsigned cnt, double price):
        bookNo(s), units_sold(cnt), revenue(cnt*price) { }
    // 其余构造函数全都委托给另一个构造函数
    Sales_data(): Sales_data("", 0, 0) {}
    Sales_data(std::string s): Sales_data(s, 0, 0) {}
    Sales_data(std::istream &is): Sales_data()
        { read(is, *this); }
    // 其他成员与之前的版本一致
};
```

在这个 `Sales_data` 类中, 除了一个构造函数外其他的都委托了它们的工作。第一个构造函数接受三个实参, 使用这些实参初始化数据成员, 然后结束工作。我们定义默认构造函数令其使用三参数的构造函数完成初始化过程, 它也无须执行其他任务, 这一点从空的构造函数体能看得出来。接受一个 `string` 的构造函数同样委托给了三参数的版本。

接受 `istream&` 的构造函数也是委托构造函数, 它委托给了默认构造函数, 默认构造函数又接着委托给三参数构造函数。当这些受委托的构造函数执行完后, 接着执行 `istream&` 构造函数体的内容。它的构造函数体调用 `read` 函数读取给定的 `istream`。

当一个构造函数委托给另一个构造函数时, 受委托的构造函数的初始值列表和函数体被依次执行。在 `Sales_data` 类中, 受委托的构造函数体恰好是空的。假如函数体包含有代码的话, 将先执行这些代码, 然后控制权才会交还给委托者的函数体。

7.5.2 节练习

练习 7.41: 使用委托构造函数重新编写你的 `Sales_data` 类, 给每个构造函数体添加一条语句, 令其一旦执行就打印一条信息。用各种可能的方式分别创建 `Sales_data` 对象, 认真研究每次输出的信息直到你确实理解了委托构造函数的执行顺序。

练习 7.42: 对于你在练习 7.40 (参见 7.5.1 节, 第 261 页) 中编写的类, 确定哪些构造函数可以使用委托。如果可以的话, 编写委托构造函数。如果不可以, 从抽象概念列表中重新选择一个你认为可以使用委托构造函数的, 为挑选出的这个概念编写类定义。



7.5.3 默认构造函数的作用

293

当对象被默认初始化或值初始化时自动执行默认构造函数。默认初始化在以下情况下发生:

- 当我们在块作用域内不使用任何初始值定义一个非静态变量 (参见 2.2.1 节, 第 39 页) 或者数组时 (参见 3.5.1 节, 第 101 页)。
- 当一个类本身含有类类型的成员且使用合成的默认构造函数时 (参见 7.1.4 节, 第 235 页)。
- 当类类型的成员没有在构造函数初始值列表中显式地初始化时 (参见 7.1.4 节, 第 237 页)。

值初始化在以下情况下发生:

- 在数组初始化的过程中如果我们提供的初始值数量少于数组的大小时 (参见 3.5.1 节, 第 101 页)。
- 当我们不使用初始值定义一个局部静态变量时 (参见 6.1.1 节, 第 185 页)。
- 当我们通过书写形如 `T()` 的表达式显式地请求值初始化时, 其中 `T` 是类型名 (`vector` 的一个构造函数只接受一个实参用于说明 `vector` 大小 (参见 3.3.1 节, 第 88 页), 它就是使用一个这种形式的实参来对它的元素初始化器进行值初始化)。

类必须包含一个默认构造函数以便在上述情况下使用, 其中的大多数情况非常容易判断。

不那么明显的一种情况是类的某些数据成员缺少默认构造函数:

```
class NoDefault {
public:
    NoDefault(const std::string&);
    // 还有其他成员, 但是没有其他构造函数了
};

struct A {           // 默认情况下 my_mem 是 public 的 (参见 7.2 节, 第 240 页)
    NoDefault my_mem;
};

A a;                 // 错误: 不能为 A 合成构造函数

struct B {
    B() {}           // 错误: b_member 没有初始值
    NoDefault b_member;
};
```

Best
Practices

在实际中, 如果定义其他构造函数, 那么最好也提供一个默认构造函数。

使用默认构造函数

◀ 294

下面的 obj 的声明可以正常编译通过：

```
Sales_data obj(); // 正确：定义了一个函数而非对象
if (obj.isbn() == Primer_5th_ed.isbn()) // 错误：obj 是一个函数
```

但当我们试图使用 obj 时，编译器将报错，提示我们不能对函数使用成员访问运算符。问题在于，尽管我们想声明一个默认初始化的对象，obj 实际的含义却是一个不接受任何参数的函数并且其返回值是 Sales_data 类型的对象。

如果想定义一个使用默认构造函数进行初始化的对象，正确的方法是去掉对象名之后的空的括号对：

```
// 正确：obj 是个默认初始化的对象
Sales_data obj;
```



WARNING

对于 C++ 的新手程序员来说有一种常犯的错误，它们试图以如下的形式声明一个用默认构造函数初始化的对象：

```
Sales_data obj(); // 错误：声明了一个函数而非对象
Sales_data obj2; // 正确：obj2 是一个对象而非函数
```

7.5.3 节练习

练习 7.43：假定有一个名为 NoDefault 的类，它有一个接受 int 的构造函数，但是没有默认构造函数。定义类 C，C 有一个 NoDefault 类型的成员，定义 C 的默认构造函数。

练习 7.44：下面这条声明合法吗？如果不，为什么？

```
vector<NoDefault> vec(10);
```

练习 7.45：如果在上一个练习中定义的 vector 的元素类型是 C，则声明合法吗？为什么？

练习 7.46：下面哪些论断是不正确的？为什么？

- (a) 一个类必须至少提供一个构造函数。
- (b) 默认构造函数是参数列表为空的构造函数。
- (c) 如果对于类来说不存在有意义的默认值，则类不应该提供默认构造函数。
- (d) 如果类没有定义默认构造函数，则编译器将为其生成一个并把每个数据成员初始化成相应类型的默认值。

7.5.4 隐式的类类型转换



4.11 节（第 141 页）曾经介绍过 C++ 语言在内置类型之间定义了几种自动转换规则。同样的，我们也能类定义隐式转换规则。如果构造函数只接受一个实参，则它实际上定义了转换为此类类型的隐式转换机制，有时我们把这种构造函数称作**转换构造函数**（converting constructor）。我们将在 14.9 节（第 514 页）介绍如何定义将一种类类型转换为另一种类类型的转换规则。

◀ 295



能通过一个实参调用的构造函数定义了一条从构造函数的参数类型向类类型隐式转换的规则。

在 `Sales_data` 类中, 接受 `string` 的构造函数和接受 `istream` 的构造函数分别定义了从这两种类型向 `Sales_data` 隐式转换的规则。也就是说, 在需要使用 `Sales_data` 的地方, 我们可以使用 `string` 或者 `istream` 作为替代:

```
string null_book = "9-999-99999-9";
// 构造一个临时的 Sales_data 对象
// 该对象的 units_sold 和 revenue 等于 0, bookNo 等于 null_book
item.combine(null_book);
```

在这里我们用一个 `string` 实参调用了 `Sales_data` 的 `combine` 成员。该调用是合法的, 编译器用给定的 `string` 自动创建了一个 `Sales_data` 对象。新生成的这个(临时) `Sales_data` 对象被传递给 `combine`。因为 `combine` 的参数是一个常量引用, 所以我们可以给该参数传递一个临时量。

只允许一步类类型转换

在 4.11.2 节(第 143 页)中我们指出, 编译器只会自动地执行一步类型转换。例如, 因为下面的代码隐式地使用了两种转换规则, 所以它是错误的:

```
// 错误: 需要用户定义两种转换:
// (1) 把 "9-999-99999-9" 转换成 string
// (2) 再把这个(临时的) string 转换成 Sales_data
item.combine("9-999-99999-9");
```

如果我们想完成上述调用, 可以显式地把字符串转换成 `string` 或者 `Sales_data` 对象:

```
// 正确: 显式地转换成 string, 隐式地转换成 Sales_data
item.combine(string("9-999-99999-9"));
// 正确: 隐式地转换成 string, 显式地转换成 Sales_data
item.combine(Sales_data("9-999-99999-9"));
```

类类型转换不是总有效

是否需要从 `string` 到 `Sales_data` 的转换依赖于我们对用户使用该转换的看法。在此例中, 这种转换可能是对的。`null_book` 中的 `string` 可能表示了一个不存在的 ISBN 编号。

另一个是从 `istream` 到 `Sales_data` 的转换:

```
// 使用 istream 构造函数创建一个函数传递给 combine
item.combine(cin);
```

这段代码隐式地把 `cin` 转换成 `Sales_data`, 这个转换执行了接受一个 `istream` 的 `Sales_data` 构造函数。该构造函数通过读取标准输入创建了一个(临时的) `Sales_data` 对象, 随后将得到的对象传递给 `combine`。

`Sales_data` 对象是个临时量(参见 2.4.1 节, 第 54 页), 一旦 `combine` 完成我们就不能再访问它了。实际上, 我们构建了一个对象, 先将它的值加到 `item` 中, 随后将其丢弃。

抑制构造函数定义的隐式转换

在要求隐式转换的程序上下文中，我们可以通过将构造函数声明为 **explicit** 加以阻止：

```
class Sales_data {
public:
    Sales_data() = default;
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    explicit Sales_data(const std::string &s): bookNo(s) { }
    explicit Sales_data(std::istream&);
    // 其他成员与之前的版本一致
};
```

此时，没有任何构造函数能用于隐式地创建 `Sales_data` 对象，之前的两种用法都无法通过编译：

```
item.combine(null_book);    // 错误：string 构造函数是 explicit 的
item.combine(cin);          // 错误：istream 构造函数是 explicit 的
```

关键字 `explicit` 只对一个实参的构造函数有效。需要多个实参的构造函数不能用于执行隐式转换，所以无须将这些构造函数指定为 `explicit` 的。只能在类内声明构造函数时使用 `explicit` 关键字，在类外部定义时不应重复：

```
// 错误：explicit 关键字只允许出现在类内的构造函数声明处
explicit Sales_data::Sales_data(istream& is)
{
    read(is, *this);
}
```

explicit 构造函数只能用于直接初始化

发生隐式转换的一种情况是当我们执行拷贝形式的初始化时（使用=）（参见 3.2.1 节，第 76 页）。此时，我们只能使用直接初始化而不能使用 `explicit` 构造函数：

```
Sales_data item1(null_book);    // 正确：直接初始化
// 错误：不能将 explicit 构造函数用于拷贝形式的初始化过程
Sales_data item2 = null_book;
```



当我们用 `explicit` 关键字声明构造函数时，它将只能以直接初始化的形式使用（参见 3.2.1 节，第 76 页）。而且，编译器将不会在自动转换过程中使用该构造函数。

为转换显式地使用构造函数

297

尽管编译器不会将 `explicit` 的构造函数用于隐式转换过程，但是我们可以使用这样的构造函数显式地强制进行转换：

```
// 正确：实参是一个显式构造的 Sales_data 对象
item.combine(Sales_data(null_book));
// 正确：static_cast 可以使用 explicit 的构造函数
item.combine(static_cast<Sales_data>(cin));
```

在第一个调用中，我们直接使用 `Sales_data` 的构造函数，该调用通过接受 `string` 的

构造函数创建了一个临时的 `Sales_data` 对象。在第二个调用中，我们使用 `static_cast`（参见 4.11.3 节，第 145 页）执行了显式的而非隐式的转换。其中，`static_cast` 使用 `istream` 构造函数创建了一个临时的 `Sales_data` 对象。

标准库中含有显式构造函数的类

我们用过的一些标准库中的类含有单参数的构造函数：

- 接受一个单参数的 `const char*` 的 `string` 构造函数（参见 3.2.1 节，第 76 页）不是 `explicit` 的。
- 接受一个容量参数的 `vector` 构造函数（参见 3.3.1 节，第 87 页）是 `explicit` 的。

7.5.4 节练习

练习 7.47: 说明接受一个 `string` 参数的 `Sales_data` 构造函数是否应该是 `explicit` 的，并解释这样做的优缺点。

练习 7.48: 假定 `Sales_data` 的构造函数不是 `explicit` 的，则下述定义将执行什么样的操作？

```
string null_isbn("9-999-99999-9");
Sales_data item1(null_isbn);
Sales_data item2("9-999-99999-9");
```

如果 `Sales_data` 的构造函数是 `explicit` 的，又会发生什么呢？

练习 7.49: 对于 `combine` 函数的三种不同声明，当我们调用 `i.combine(s)` 时分别发生什么情况？其中 `i` 是一个 `Sales_data`，而 `s` 是一个 `string` 对象。

- `Sales_data &combine(Sales_data);`
- `Sales_data &combine(Sales_data&);`
- `Sales_data &combine(const Sales_data&) const;`

练习 7.50: 确定在你的 `Person` 类中是否有一些构造函数应该是 `explicit` 的。

练习 7.51: `vector` 将其单参数的构造函数定义成 `explicit` 的，而 `string` 则不是，你觉得原因何在？



7.5.5 聚合类

298

聚合类（aggregate class）使得用户可以直接访问其成员，并且具有特殊的初始化语法形式。当一个类满足如下条件时，我们说它是聚合的：

- 所有成员都是 `public` 的。
- 没有定义任何构造函数。
- 没有类内初始值（参见 2.6.1 节，第 64 页）。
- 没有基类，也没有 `virtual` 函数，关于这部分知识我们将在第 15 章详细介绍。

例如，下面的类是一个聚合类：

```
struct Data {
    int ival;
    string s;
};
```

我们可以提供一个花括号括起来的成员初始值列表，并用它初始化聚合类的数据成员：

```
// val1.ival = 0; val1.s = string("Anna")
Data val1 = { 0, "Anna" };
```

初始值的顺序必须与声明的顺序一致，也就是说，第一个成员的初始值要放在第一个，然后是第二个，以此类推。下面的例子是错误的：

```
// 错误：不能使用"Anna"初始化 ival，也不能使用 1024 初始化 s
Data val2 = { "Anna" , 1024 };
```

与初始化数组元素的规则（参见 3.5.1 节，第 101 页）一样，如果初始值列表中的元素个数少于类的成员数量，则靠后的成员被值初始化（参见 3.5.1 节，第 101 页）。初始值列表的元素个数绝对不能超过类的成员数量。

值得注意的是，显式地初始化类的对象的成员存在三个明显的缺点：

- 要求类的所有成员都是 `public` 的。
- 将正确初始化每个对象的每个成员的重任交给了类的用户（而非类的作者）。因为用户很容易忘掉某个初始值，或者提供一个不恰当的初始值，所以这样的初始化过程冗长乏味且容易出错。
- 添加或删除一个成员之后，所有的初始化语句都需要更新。

7.5.5 节练习

299

练习 7.52：使用 2.6.1 节（第 64 页）的 `Sales_data` 类，解释下面的初始化过程。如果存在问题，尝试修改它。

```
Sales_data item = {"978-0590353403", 25, 15.99};
```

7.5.6 字面值常量类



在 6.5.2 节（第 214 页）中我们提到过 `constexpr` 函数的参数和返回值必须是字面值类型。除了算术类型、引用和指针外，某些类也是字面值类型。和其他类不同，字面值类型的类可能含有 `constexpr` 函数成员。这样的成员必须符合 `constexpr` 函数的所有要求，它们是隐式 `const` 的（参见 7.1.2 节，第 231 页）。

数据成员都是字面值类型的聚合类（参见 7.5.5 节，第 266 页）是字面值常量类。如果一个类不是聚合类，但它符合下述要求，则它也是一个字面值常量类：

- 数据成员都必须是字面值类型。
- 类必须至少含有一个 `constexpr` 构造函数。
- 如果一个数据成员含有类内初始值，则内置类型成员的初始值必须是一条常量表达式（参见 2.4.4 节，第 58 页）；或者如果成员属于某种类类型，则初始值必须使用成员自己的 `constexpr` 构造函数。
- 类必须使用析构函数的默认定义，该成员负责销毁类的对象（参见 7.1.5 节，第 239 页）。

`constexpr` 构造函数

尽管构造函数不能是 `const` 的（参见 7.1.4 节，第 235 页），但是字面值常量类的构造函数可以是 `constexpr`（参见 6.5.2 节，第 213 页）函数。事实上，一个字面值常量类必须至少提供一个 `constexpr` 构造函数。

C++
11

constexpr 构造函数可以声明成=default (参见 7.1.4 节, 第 237 页) 的形式 (或者是删除函数的形式, 我们将在 13.1.6 节 (第 449 页) 介绍相关知识)。否则, constexpr 构造函数就必须既符合构造函数的要求 (意味着不能包含返回语句), 又符合 constexpr 函数的要求 (意味着它能拥有的唯一可执行语句就是返回语句 (参见 6.5.2 节, 第 214 页))。综合这两点可知, constexpr 构造函数体一般来说应该是空的。我们通过前置关键字 constexpr 就可以声明一个 constexpr 构造函数了:

```
class Debug {
public:
    constexpr Debug(bool b = true): hw(b), io(b), other(b) { }
    constexpr Debug(bool h, bool i, bool o):
        hw(h), io(i), other(o) { }
    constexpr bool any() { return hw || io || other; }
    void set_io(bool b) { io = b; }
    void set_hw(bool b) { hw = b; }
    void set_other(bool b) { other = b; }
private:
    bool hw;                // 硬件错误, 而非 IO 错误
    bool io;                // IO 错误
    bool other;             // 其他错误
};
```

300

constexpr 构造函数必须初始化所有数据成员, 初始值或者使用 constexpr 构造函数, 或者是一条常量表达式。

constexpr 构造函数用于生成 constexpr 对象以及 constexpr 函数的参数或返回类型:

```
constexpr Debug io_sub(false, true, false);    // 调试 IO
if (io_sub.any())                             // 等价于 if(true)
    cerr << "print appropriate error messages" << endl;
constexpr Debug prod(false);                 // 无调试
if (prod.any())                              // 等价于 if(false)
    cerr << "print an error message" << endl;
```

7.5.6 节练习

练习 7.53: 定义你自己的 Debug。

练习 7.54: Debug 中以 set_ 开头的成员应该被声明成 constexpr 吗? 如果不, 为什么?

练习 7.55: 7.5.5 节 (第 266 页) 的 Data 类是字面值常量类吗? 请解释原因。

7.6 类的静态成员

有的时候类需要它的一些成员与类本身直接相关, 而不是与类的各个对象保持关联。例如, 一个银行账户类可能需要一个数据成员来表示当前的基准利率。在此例中, 我们希望利率与类关联, 而非与类的每个对象关联。从实现效率的角度来看, 没必要每个对象都存储利率信息。而且更加重要的是, 一旦利率浮动, 我们希望所有的对象都能使用新值。